

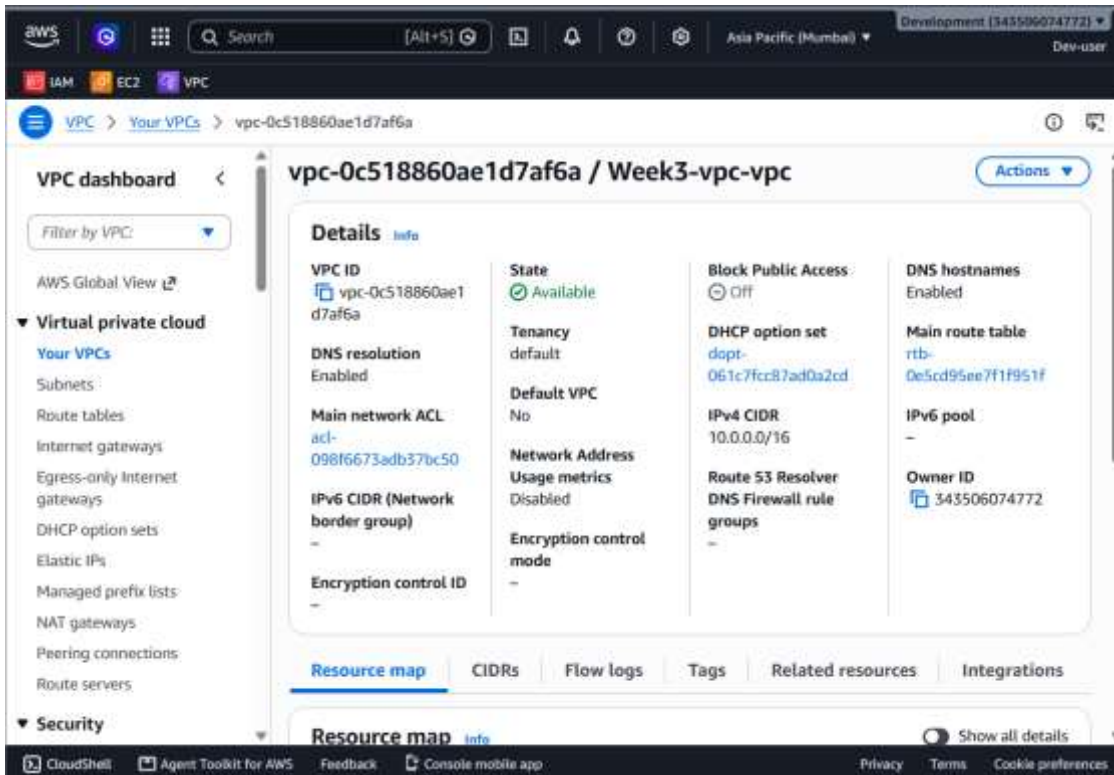
AWS Internship – Week 3

Networking, Databases & Serverless Computing

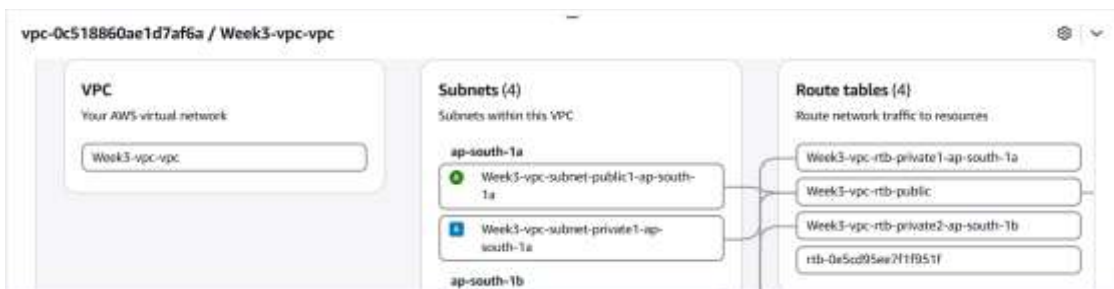
1. Created a VPC with Public and Private Subnets

Steps:

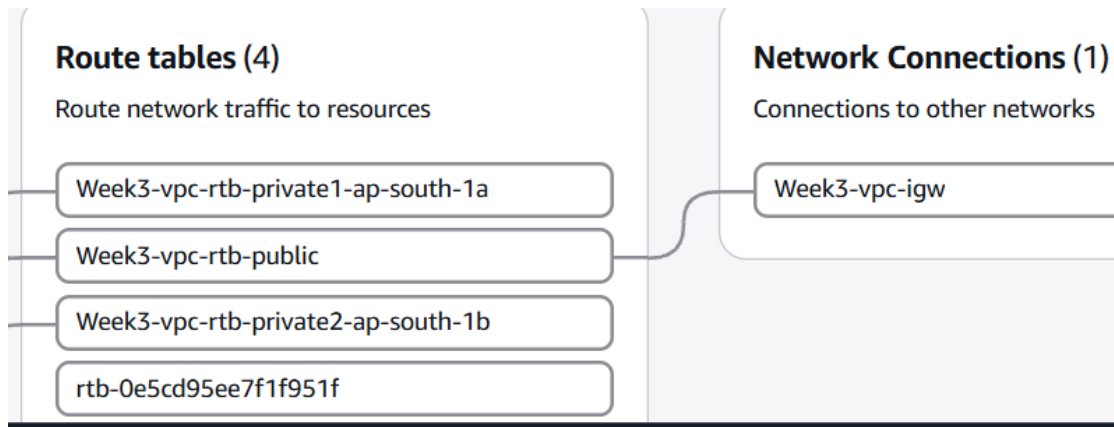
- Created a custom VPC ("Week3-vpc-vpc") using the "VPC and more" wizard, spanning 2 Availability Zones (ap-south-1a and ap-south-1b).
- The wizard automatically provisioned 2 public subnets and 2 private subnets, along with an Internet Gateway and route tables.
- Verified the public route table ("Week3-vpc-rtb-public") was connected to the Internet Gateway ("Week3-vpc-igw"), while the private route tables had no internet route — keeping those subnets isolated.



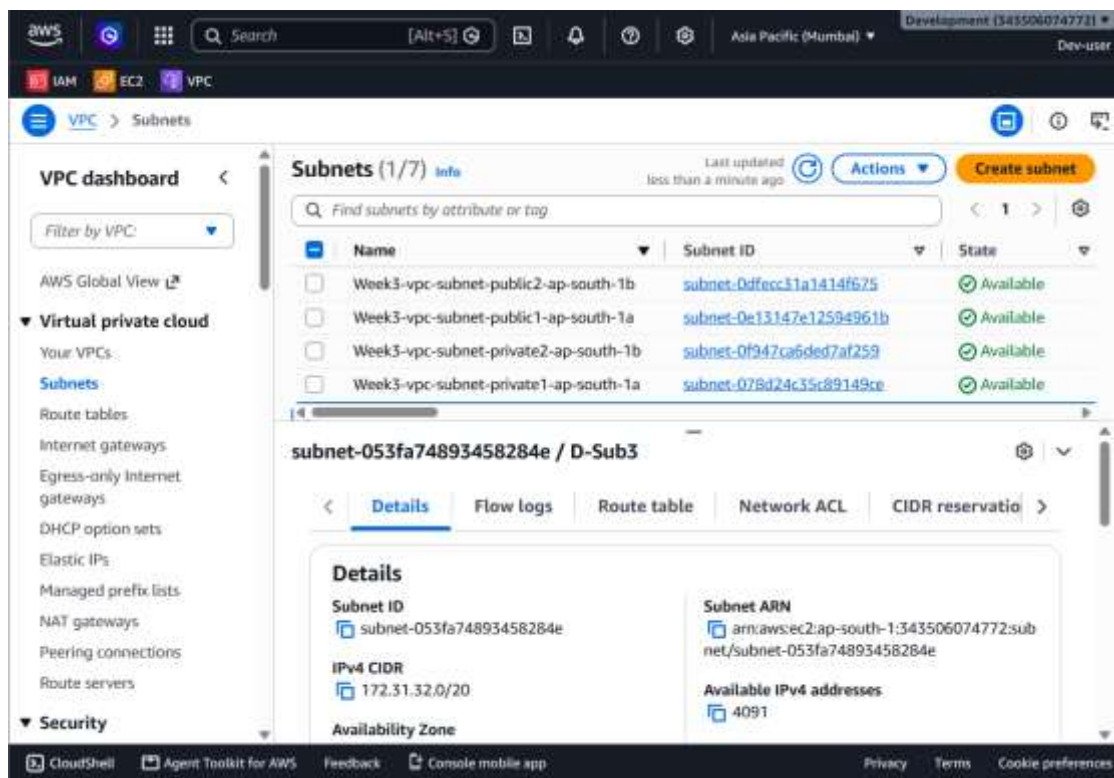
VPC details: CIDR 10.0.0.0/16, DNS resolution enabled, state Available



VPC resource map showing 4 subnets across 2 Availability Zones and 4 route tables



Route tables connected to the Internet Gateway (public route table only)



All 4 subnets (2 public, 2 private) listed as Available

2. Launched EC2 Instances Inside the VPC

Steps:

- Launched "Week3-Public-Ec2" in a public subnet with an auto-assigned public IP, and security group rules allowing SSH (port 22) and HTTP (port 80).
- Launched "Week3-Private-ec2" in a private subnet with no public IP, to demonstrate network isolation.
- Connected successfully to the public instance via EC2 Instance Connect after adjusting the security group's SSH rule to allow the current IP.
- Attempted to connect to the private instance and confirmed it was unreachable — AWS explicitly reported "No public IPv4 or IPv6 address assigned" and "Instance is not in public subnet," confirming correct isolation by design.

The screenshot shows the AWS Management Console 'Instances' page. The left sidebar lists navigation options under 'EC2' and 'Images'. The main content area shows 'Instances (2)' with a table listing two instances: 'Week3-Private...' and 'Week3-Public...'. Both are in a 'Running' state. Below the table, there are three summary cards: 'Instance alarm states' (In alarm), 'Service health' (EC2 service status: Operating normally), and 'Health events affecting EC2 instances'.

Name	Instance ID	Instance state	Instance type	Stats
Week3-Private...	i-0b02e459b888d8348	Running	t3.micro	3/
Week3-Public...	i-0641ca91f54e4cdae	Running	t3.micro	3/

Both EC2 instances running inside the custom VPC

The screenshot shows the 'Instances (1/3)' page with the 'Week3-Public...' instance selected. Below the table, the 'Instance summary' card provides details for instance 'i-0641ca91f54e4cdae'. It shows the instance is 'Running' and provides its public IPv4 address (52.66.189.140), private IPv4 address (10.0.3.69), and public DNS name (ec2-52-66-189-140.ap-south-1.compute.amazonaws.com).

Name	Instance ID	Instance state	Instance type
Week3-Public...	i-0641ca91f54e4cdae	Running	t3.micro
Week3-web-srv	i-04f6061588a2511ba	Terminated	t3.micro

Instance summary

Instance ID: i-0641ca91f54e4cdae

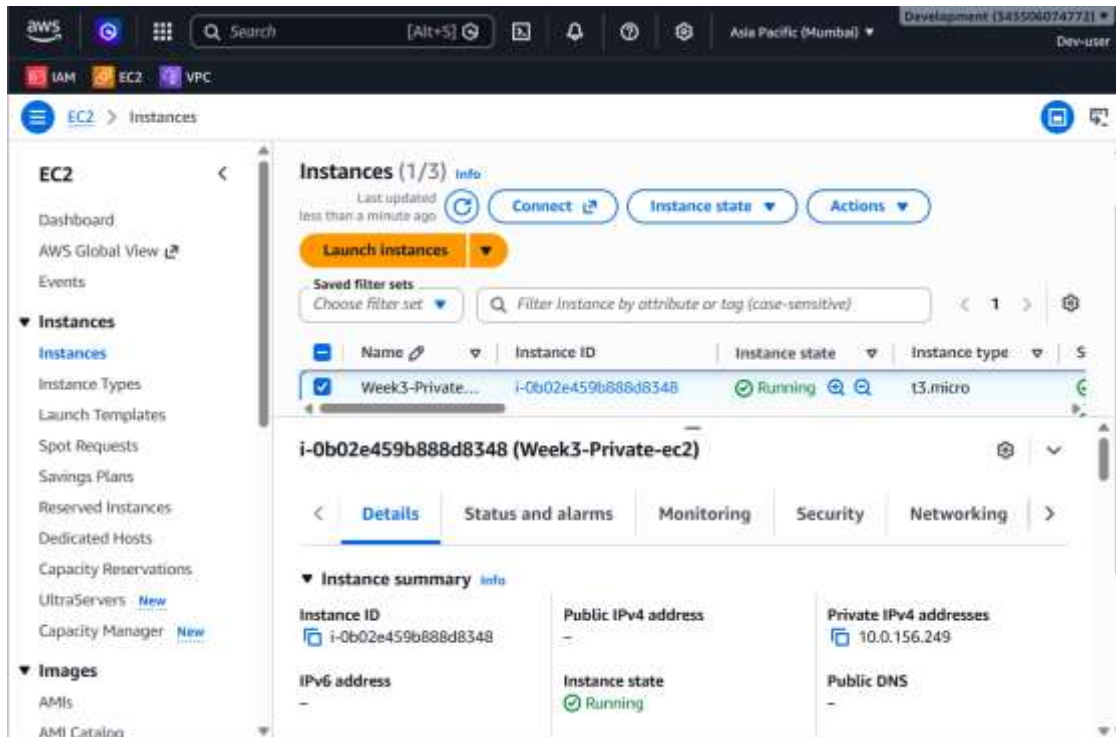
Public IPv4 address: 52.66.189.140 | [open address](#)

Private IPv4 addresses: 10.0.3.69

Instance state: Running

Public DNS: ec2-52-66-189-140.ap-south-1.compute.amazonaws.com | [open](#)

Public instance details — public IP 52.66.189.140 assigned

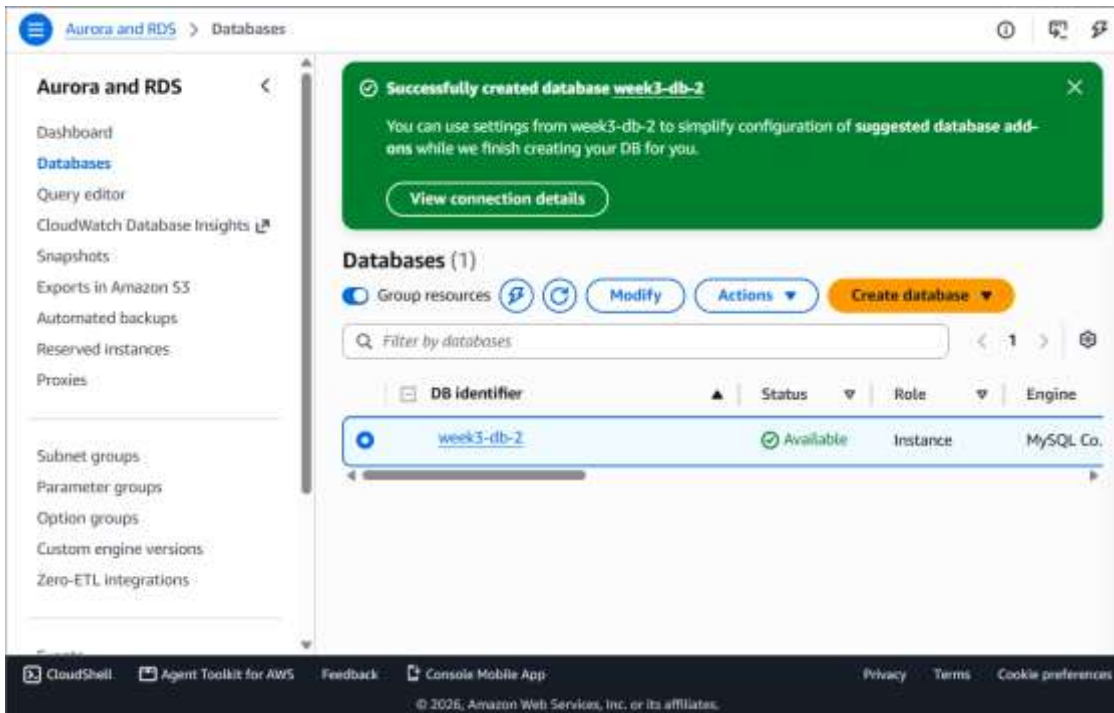


Private instance details — no public IP assigned, as expected

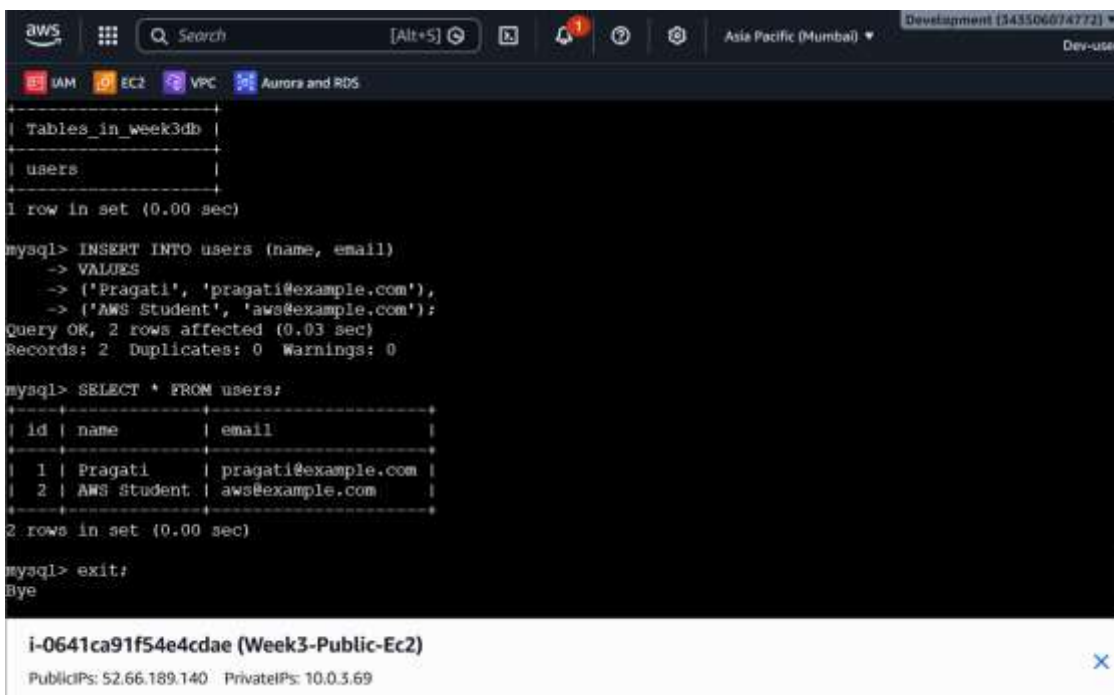
3. Created and Connected an RDS MySQL Database

Steps:

- Created an RDS MySQL instance ("week3-db-2") inside the custom VPC, using free-tier eligible settings (db.t3.micro, 20 GiB gp3 storage, Single-AZ, Dev/Test template).
- Set Public access to "No," ensuring the database could only be reached from within the VPC, not from the internet.
- Created a new dedicated security group ("Week3-db-sg") and added an inbound rule allowing MySQL traffic (port 3306) only from the EC2 instances' security group.
- Installed the MySQL client on the public EC2 instance and connected to the database using its endpoint, username, and password.
- Created a "users" table, inserted sample records, and successfully queried them back, confirming full connectivity between EC2 and RDS inside the VPC.



RDS MySQL instance "week3-db-2" successfully created and Available



Connected to RDS from EC2, created a table, inserted and queried data successfully

4. Built a Lambda Function

Steps:

- Created a Lambda function ("Week3-fun") using the Python runtime, authored from scratch.
- Wrote a custom handler that returns a personalized JSON greeting, reading an optional "name" query parameter with a default fallback value.
- Resolved an initial syntax error and a leftover/duplicate code issue by fully clearing and re-pasting the function body before redeploying.
- Deployed and tested the function directly in the Lambda console, confirming correct JSON output.

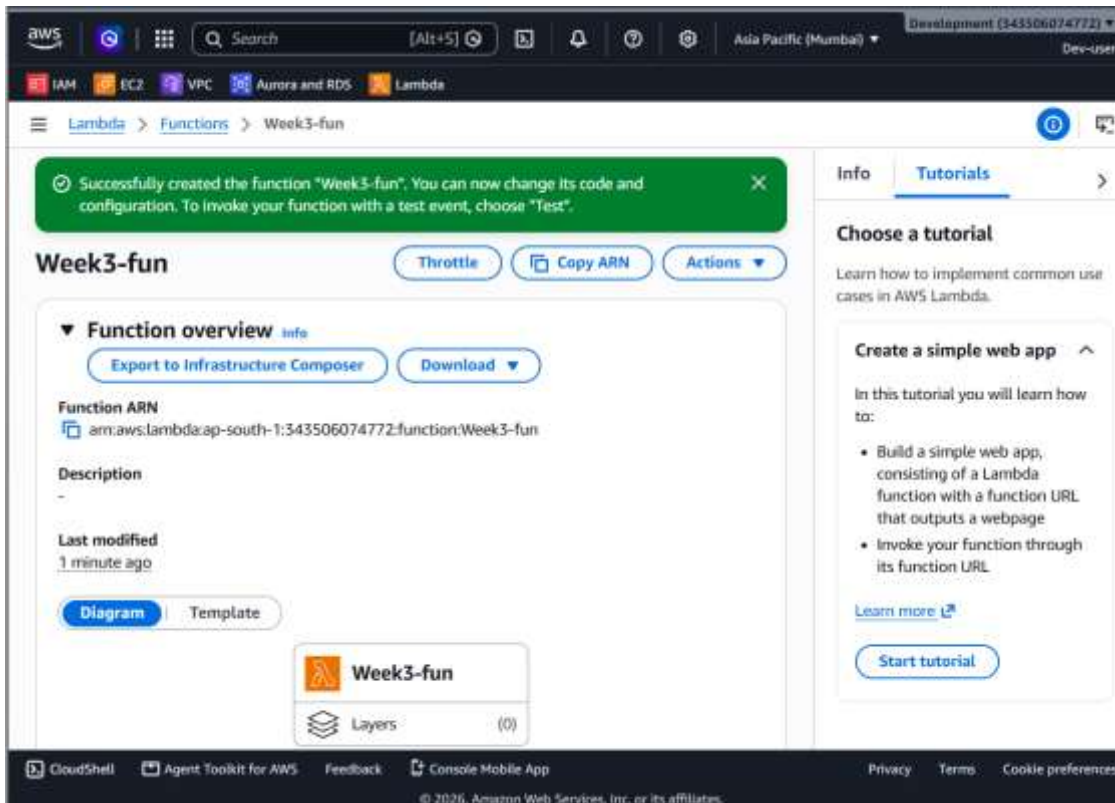
```
import json
```

```
def lambda_handler(event, context):
```

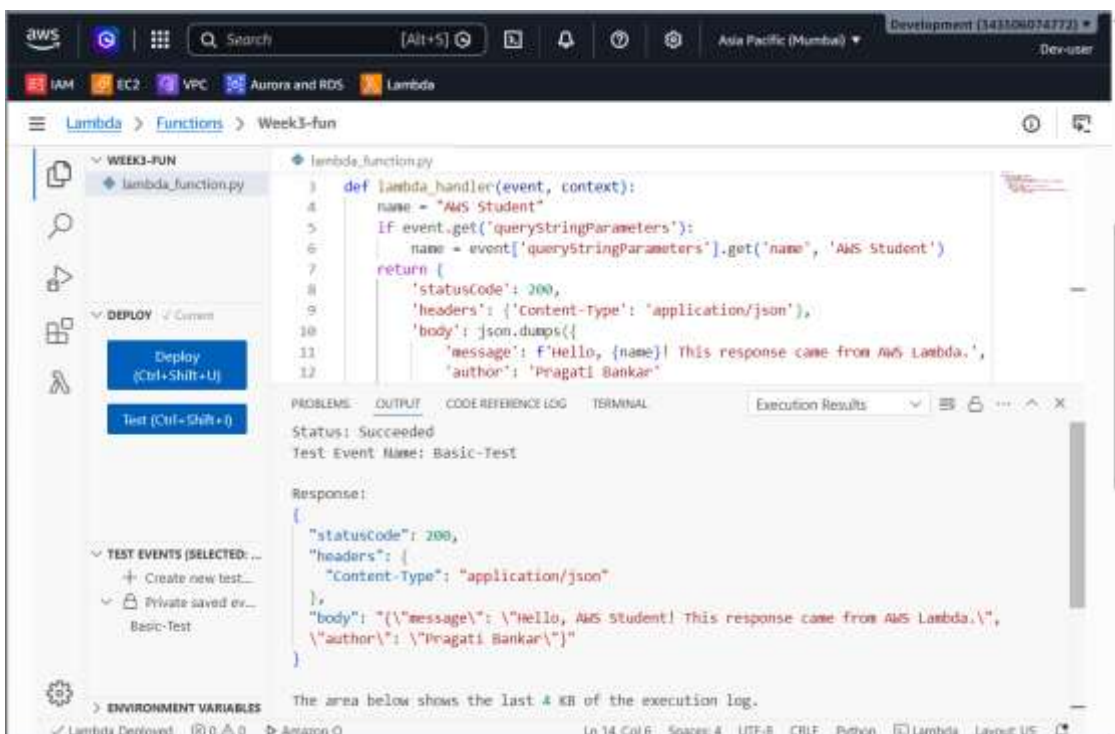
```

name = "AWS Student"
if event.get('queryStringParameters'):
    name = event['queryStringParameters'].get('name', 'AWS Student')
return {
    'statusCode': 200,
    'headers': {'Content-Type': 'application/json'},
    'body': json.dumps({
        'message': f'Hello, {name}! This response came from AWS Lambda.',
        'author': 'Pragati Bankar'
    })
}

```



Lambda function "Week3-fun" successfully created

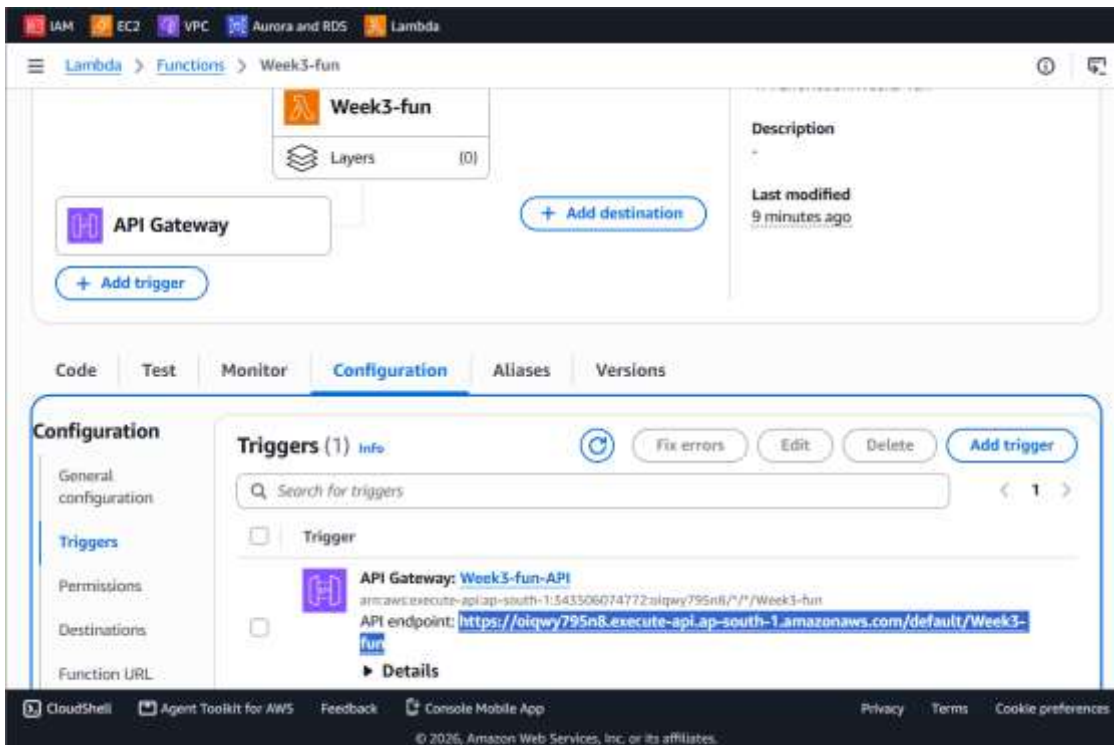


Function deployed and tested successfully in the console — correct JSON response returned

5. Connected Lambda to API Gateway and Tested It

Steps:

- Added an API Gateway (HTTP API) trigger to the Lambda function, with open security for testing purposes.
- Received a public API endpoint URL from AWS.
- Opened the endpoint URL directly in a browser and received the same JSON response as the console test, confirming the function was successfully exposed as a public API.
- Tested the optional "name" query parameter by appending it to the URL, confirming the function correctly reads and uses input from the request.



API Gateway trigger added, showing the public API endpoint URL



Live JSON response from the public API endpoint, viewed directly in the browser

6. CloudWatch Metrics for EC2

The EC2 instances' Monitoring tab was reviewed before the instances were terminated, showing AWS's default CloudWatch metrics (CPU utilization, network in/out, disk activity) automatically graphed with no additional setup required. Since the instances were deleted as part of routine cleanup to avoid unnecessary running resources, a live screenshot of the CloudWatch dashboard was not captured for this report, but the metrics were confirmed to populate correctly in the console during the exercise.

Learning Outcome

This week connected networking, databases, and serverless computing into a single working system. Building a custom VPC with public and private subnets made the difference between "reachable" and "isolated" resources concrete — the private EC2 instance and the private RDS database were both unreachable from the internet by design, not by accident, and AWS's own error messages confirmed this. Connecting to RDS from an EC2 instance inside the same VPC, and successfully running SQL queries, tied directly back to earlier MySQL experience, just now running on managed cloud infrastructure instead of a local machine. Building and deploying a Lambda function, then exposing it through API Gateway, demonstrated a completely different model: no server to patch,

size, or manage at all, just code that runs on demand and is billed only when invoked. Troubleshooting both the recurring IAM permissions boundary issue and a Lambda code deployment error reinforced that methodical debugging — checking as root, clearing and redeploying cleanly — is a normal and necessary part of cloud work, not a sign of doing something wrong.